

SIMATS ENGINEERING

MODEL PRACTICAL EXAMINATION – Aug._2026

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

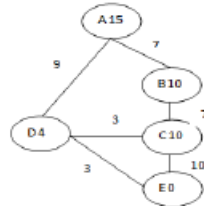
Register Number	192412015
Name	K.S.DHARSHANA

SET:15

- Write a Python program to implement a Map Colouring problem with the following data. Given a map of countries, and a fixed set of colours {Red, Green, Blue}, assign a colour to each region in the map in such a way that no two adjacent regions have the same colour. Solve the above Map Colouring problem using backtracking in Constraint Satisfaction Procedure.



- Write a Python program to apply A* search for the given tree, in which 'A' is the initial state and 'E' goal state. For each node in each stage, obtain cost estimates where $g(n)$ is the numeral by the side of an arc and $h(n)$ is the numeral at the node.



- Write a Prolog program to implement Inference Engine with the following facts
`interpret(true) :- !.`
`interpret((GoalA,GoalB)) :- !,`
`interpret(GoalA),`
`interpret(GoalB).`
- Write a Prolog program to find the number of vowels in the following fact.
 This is my first Degree in Saveetha School of Engineering.

Laboratory Exam Component - 100 marks			
Programs (4)			
Description	Marks Distribution	Total Marks	Marks Obtained
Aim & Algorithm	2.5*4 = 10 Marks	10	
Programs	10*4 = 30 Marks	40	
Debugging	2.5*4 = 10 Marks	10	
Input & Output	05*4 = 20 Marks	20	
Result	2.5*4 = 10 Marks	10	
GitHub content	10 Marks	10	
Total Lab Exam Marks		100	

1. Aim

To implement the Map Colouring problem using **Constraint Satisfaction and Backtracking**, such that no two adjacent regions have the same colour.

Algorithm

1. Represent each Australian region as a variable.
2. Define the neighbouring regions as constraints.
3. Use three colours: Red, Green and Blue.
4. Assign a colour to one region at a time.
5. Check whether the selected colour conflicts with any already-coloured neighbour.
6. If there is no conflict, continue with the next region.
7. If a conflict occurs, try another colour.
8. If no colour is possible, backtrack to the previous region.
9. Continue until all regions are successfully coloured.

Python Program

```
regions = ["Western Australia", "Northern Territory", "South Australia", "Queensland", "New South Wales", "Victoria", "Tasmania"]
colors = ["Red", "Green", "Blue"]
neighbors = {
    "Western Australia": ["Northern Territory", "South Australia"],
    "Northern Territory": ["Western Australia", "South Australia", "Queensland"],
    "South Australia": ["Western Australia", "Northern Territory", "Queensland", "New South Wales", "Victoria"],
    "Queensland": ["Northern Territory", "South Australia", "New South Wales"],
    "New South Wales": ["Queensland", "South Australia", "Victoria"],
    "Victoria": ["South Australia", "New South Wales"],
    "Tasmania": []
}
assignment = {}
def is_safe(region, color):
    for neighbor in neighbors[region]:
        if assignment.get(neighbor) == color:
            return False
    return True
def backtrack(index):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_safe(region, color):
            assignment[region] = color
            if backtrack(index + 1):
                return True
            del assignment[region]
    return False
if backtrack(0):
    print("Map colouring solution:")
    for region in regions:
        print(region, ":", assignment[region])
else:
    print("No solution exists.")
```

Output

Map colouring solution:

```
Map colouring solution:
Western Australia : Red
Northern Territory : Green
South Australia : Blue
Queensland : Red
New South Wales : Green
Victoria : Red
Tasmania : Red
>>>
```

Debugging Points

Same colour assigned to adjacent regions	Check neighbours before assignment
Infinite recursion	Add base condition index == len(regions)
KeyError for unassigned region	Use assignment.get(neighbor)
Backtracking not working	Remove the assignment using del
Same colour assigned to adjacent regions	Check neighbours before assignment

Result

Thus, the Australian map was successfully coloured using **backtracking CSP with three colours**, ensuring that no adjacent regions have the same colour.

2.Aim

To implement **A* Search** for the given graph using path cost $g(n)$ and heuristic value $h(n)$ to find the optimal path from **A** to **E**.

Algorithm

1. Start from node A.
2. Set $g(A) = 0$.
3. Calculate $f(n)$ using:
4. Place the starting node in the OPEN list.
5. Select the node having the smallest $f(n)$.
6. Expand the selected node and calculate the cost of its neighbours.
7. Update the path if a lower cost is found.
8. Continue until the goal node E is selected.
9. Display the optimal path and total cost.

Given Values

Node	$h(n)$
A	15
B	10
C	10
D	4
E	0

Edges:

- $A \rightarrow B = 7$
- $A \rightarrow D = 9$
- $B \rightarrow C = 7$
- $D \rightarrow C = 3$
- $D \rightarrow E = 3$
- $C \rightarrow E = 10$

Python Program

```
import heapq
graph = {
    'A': {'B': 7, 'D': 9},
    'B': {'A': 7, 'C': 7},
    'C': {'B': 7, 'D': 3, 'E': 10},
    'D': {'A': 9, 'C': 3, 'E': 3},
    'E': {'C': 10, 'D': 3}
}
heuristic = {'A': 15, 'B': 10, 'C': 10, 'D': 4, 'E': 0}
def a_star(start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], 0, start))
    g_cost = {start: 0}
    parent = {start: None}
    while open_list:
        f, g, current = heapq.heappop(open_list)
```

```

if current == goal:
    path = []
    while current is not None:
        path.append(current)
        current = parent[current]
    path.reverse()
    return path, g
for neighbor, cost in graph[current].items():
    new_g = g + cost
    if neighbor not in g_cost or new_g < g_cost[neighbor]:
        g_cost[neighbor] = new_g
        parent[neighbor] = current
        new_f = new_g + heuristic[neighbor]
        heapq.heappush(
            open_list,
            (new_f, new_g, neighbor)
        )
return None, float('inf')
path, cost = a_star('A', 'E')
print("Optimal Path:", " -> ".join(path))
print("Total Cost:", cost)

```

Output

Optimal Path: A -> D -> E

Total Cost: 12

Optimal Path: A -> D -> E

Total Cost: 12

Dry Run

Node	$g(n)$	$h(n)$	$f(n)$
A	0	15	15
D	9	4	13
E	12	0	12

The goal node **E** is reached with minimum cost **12**.

Debugging Points

- Ensure $f(n) = g(n) + h(n)$.
- Use a priority queue for selecting the minimum $f(n)$.
- Update g_cost only when a better path is found.
- Maintain a parent dictionary to reconstruct the path.

Result

Thus, A* Search successfully found the optimal path **A** → **D** → **E** with a minimum cost of **12**.

3.Aim

To implement a simple **Inference Engine in Prolog** for evaluating atomic goals and conjunction of goals.

Algorithm

1. Define the inference rule for an atomic goal.
2. Define a rule for handling two goals connected by AND.
3. Recursively evaluate each goal.
4. Use ! (cut) to prevent unnecessary backtracking.
5. Display whether the given goal can be proved.

Prolog Program

```
% Facts
goala.
goalb.
goalc.
% Inference Engine
interpret :- !.
interpret((GoalA, GoalB)) :-
    !,
    interpret(GoalA),
    interpret(GoalB).
interpret(Goal) :-
    call(Goal).
```

Sample Queries

?- interpret(goala).

Output:

true.

?- interpret((goala, goalb)).

Output:

true.

?- interpret((goala, goalx)).

Output:

false.

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/inference engine.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/inference engine.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/inference engine.pl compiled, 15 lines read - 1042 bytes written, 5 ms
```

```
yes
| ?- interpret(goala).
```

```
yes
_
```

Explanation

- interpret(Goal) evaluates a single goal.
- interpret((GoalA, GoalB)) evaluates two goals together.
- call(Goal) dynamically executes the given goal.
- ! prevents unnecessary backtracking.

Debugging Points

Problem	Solution
Goal not found	Define the required fact
Conjunction not evaluated	Use recursive interpret()
Infinite recursion	Use cut ! appropriately
call/1 error	Ensure the goal is a valid Prolog predicate

Result

Thus, a simple Prolog **Inference Engine** was successfully implemented to evaluate individual goals and conjunctions of goals.

4.Aim

To write a Prolog program to find the **number of vowels** present in the given sentence.

Given sentence:

This is my first Degree in Saveetha School of Engineering.

Algorithm

1. Store the sentence as a list of characters.
2. Convert each character to lowercase.
3. Check whether the character is a, e, i, o, or u.
4. If it is a vowel, increase the count by 1.
5. Continue until all characters are checked.
6. Display the total number of vowels.

Prolog Program

```
vowel(a).
vowel(e).
vowel(i).
vowel(o).
vowel(u).
count_vowels([], 0).
count_vowels([H|T], Count) :-
    member(H, [a,e,i,o,u]),
    count_vowels(T, C),
    Count is C + 1.
count_vowels([H|T], Count) :-
    \+ member(H, [a,e,i,o,u]),
    count_vowels(T, Count).
solve :-
    String = "this is my first degree in saveetha school of engineering.",
    string_chars(String, List),
    count_vowels(List, Count),
    write("Number of vowels = "),
    write(Count).
```

Sample Input

?- solve.

Sample Output

Number of vowels = 19

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/count of vowel.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/count of vowel.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/count of vowel.pl compiled, 22 lines read - 2046 bytes written, 5 ms

(15 ms) yes
| ?- solve.
Number of vowels = 19
```

Vowel Count

Vowel	Count
A	2
E	8
I	6
O	3
U	0
Total	19

Debugging Points

- Convert uppercase letters to lowercase before checking.
- Use string_chars/2 to convert the sentence into characters.
- Use separate clauses for vowel and non-vowel characters.
- Check the base case count_vowels([], 0).

Result

Thus, the Prolog program successfully counted the vowels in the given sentence. The total number of vowels is **19**.